PROGRAMACIÓN III · ANGULAR

Angular

Autenticación, Interceptors, Guards y Pipes

AuthService y JWT · Interceptors (middleware del cliente)
Guards de ruta · Pipes de transformación

Instituto Técnico Superior Córdoba

Programación III · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026

#	Sección
01	Flujo de autenticación: el circuito completo
02	AuthService: el servicio de autenticación
03	Interceptors: el middleware de Angular
04	Guards: proteger rutas en Angular
05	Pipes: transformar datos en el template
06	Flujo completo: todo junto
07	Resumen comparativo · Conceptos clave

01 · Flujo de autenticación: el circuito completo

Antes de hablar de interceptors o guards, necesitamos entender **el flujo completo de autenticación** entre Angular y la API .NET. Es el mismo flujo que vimos desde el backend, pero ahora lo vamos a recorrer desde el punto de vista del frontend.

El circuito tiene 4 pasos fundamentales:

Paso	Quién	Qué hace
1	**Usuario**	Completa el formulario de login (email + contraseña)
2	**Angular**	Envía POST /api/auth/login con las credenciales en el body
3	**API .NET**	Valida las credenciales y genera un JWT si son correctas
4	**Angular**	Recibe el JWT, lo guarda en localStorage y lo usa en cada request siguiente

> **Punto en común con el backend:** el paso 3 es exactamente lo que ya vimos en el material de .NET — el endpoint de login valida credenciales con la base de datos y retorna un JWT firmado. Ahora vamos a ver cómo Angular consume ese endpoint y qué hace con el token.

Una vez que Angular tiene el JWT guardado, cada request posterior a la API debe incluir el header `Authorization: Bearer eyJhbGci...`. Sin ese header, la API retorna **401 Unauthorized**. Este es el problema que los interceptors resuelven automáticamente (lo veremos en la sección 3).

02 · AuthService: el servicio de autenticación

El **AuthService** es el servicio de Angular que centraliza toda la lógica de autenticación: hacer login, guardar el token, verificar si el usuario está logueado y hacer logout. Ningún componente debería manejar el JWT directamente — siempre pasa por este servicio.

Modelo de login y respuesta

models/auth.model.ts

```
export interface LoginRequest {
    email: string;
    password: string;
}

export interface LoginResponse {
    token: string;
    expiration: string;  // fecha de expiración del JWT
}
```

El servicio completo

services/auth.service.ts

```
@Injectable({ providedIn: 'root' })
export class AuthService {
    private apiUrl = environment.apiUrl + '/auth';
    private readonly TOKEN_KEY = 'jwt_token';

    constructor(private http: HttpClient) {}

    // 1. LOGIN: envía credenciales y guarda el token
    login(credentials: LoginRequest): Observable<LoginResponse> {
        return this.http.post<LoginResponse>(
            `${this.apiUrl}/login`, credentials
        ).pipe(
            tap(response => {
                localStorage.setItem(this.TOKEN_KEY, response.token);
            })
        );
    }

    // 2. LOGOUT: elimina el token
    logout(): void {
        localStorage.removeItem(this.TOKEN_KEY);
    }

    // 3. OBTENER TOKEN: lo usa el interceptor
    getToken(): string | null {
        return localStorage.getItem(this.TOKEN_KEY);
    }

    // 4. VERIFICAR AUTENTICACIÓN: lo usa el guard
    isAuthenticated(): boolean {
        const token = this.getToken();
        return token !== null;  // versión simple
    }
}
```

> **Un único punto:** cada método del AuthService será consumido por una pieza distinta del sistema: `login()` lo usa el componente de login, `getToken()` lo usa el interceptor, `isAuthenticated()` lo usa el guard, y `logout()` lo usa el navbar.

Componente de Login

El componente de login es el formulario donde el usuario ingresa sus credenciales. Al hacer submit, llama al AuthService y si el login es exitoso, redirige al usuario a la página principal.

components/login/login.component.ts

```
@Component({
    standalone: true,
    selector: 'app-login',
    imports: [FormsModule, CommonModule],
    template: `
        <div class="login-container">
            <h2>Iniciar Sesión</h2>
            <div *ngIf="error" class="error">{{ error }}</div>
            <input type="email" [(ngModel)]="credentials.email" placeholder="Email" />
            <input type="password" [(ngModel)]="credentials.password" placeholder="Contraseña" />
            <button (click)="onLogin()">Ingresar</button>
        </div>`
})
export class LoginComponent {
    credentials: LoginRequest = { email: '', password: '' };
    error = '';

    constructor(private authService: AuthService, private router: Router) {}

    onLogin(): void {
        this.authService.login(this.credentials).subscribe({
            next: () => this.router.navigate(['/productos']),
            error: (err) => this.error = 'Credenciales inválidas'
        });
    }
}
```

Observá el patrón: el componente no sabe nada sobre JWT ni localStorage. Solo llama a `authService.login()` y reacciona al resultado. Toda la lógica de autenticación está encapsulada en el servicio.

03 · Interceptors: el middleware de Angular

¿Qué es un interceptor?

Un **interceptor** es un componente que se ubica en el pipeline de requests HTTP de Angular y procesa **cada request antes de que salga** y **cada response antes de que llegue** al componente. Es exactamente el mismo concepto que un **middleware en .NET**, pero del lado del cliente.

Concepto	.NET (Backend)	Angular (Frontend)
Qué es	Middleware	Interceptor
Dónde se ejecuta	Pipeline de ASP.NET Core	Pipeline de HttpClient
Se registra en	Program.cs con app.UseMiddleware<>()	app.config.ts con provideHttpClient()
Procesa	Cada request que entra al servidor	Cada request que sale del navegador
Caso típico	Logging, excepciones, CORS	Inyectar JWT, manejar errores HTTP

> **Analogía del restaurante:** si el middleware es el maître que revisa la reserva antes de dejarte entrar al salón, el interceptor es el mozo que agrega la propina a la cuenta antes de llevarla a la cocina. Uno filtra lo que entra, el otro prepara lo que sale.

Interceptor de autenticación (JWT)

El caso de uso más importante: agregar automáticamente el header `Authorization: Bearer <token>` a cada request que sale de Angular. Sin esto, tendrías que agregar el header manualmente en cada llamada HTTP de cada servicio.

interceptors/auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
    const authService = inject(AuthService);
    const token = authService.getToken();

    // Si hay token, clonamos el request y le agregamos el header
    if (token) {
        const clonedReq = req.clone({
            setHeaders: { Authorization: `Bearer ${token}` }
        });
        return next(clonedReq);
    }

    // Si no hay token, dejamos pasar el request sin modificar
    return next(req);
};
```

> **¿Por qué clonamos el request?** En Angular, los objetos HttpRequest son inmutables. No podés modificar un request existente — tenés que crear una copia con los cambios usando `req.clone()`.

Interceptor de errores HTTP

Otro interceptor muy común: capturar errores HTTP globalmente. En lugar de manejar el error 401 en cada servicio, lo manejás una sola vez en el interceptor. Es el equivalente al `ExceptionMiddleware` de .NET.

interceptors/error.interceptor.ts

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
    const router = inject(Router);
    const authService = inject(AuthService);

    return next(req).pipe(
        catchError(error => {
            if (error.status === 401) {
                // Token inválido o expirado -> logout y redirigir
                authService.logout();
                router.navigate(['/login']);
            }
            if (error.status === 403) {
                router.navigate(['/sin-permiso']);  // No tiene permisos
            }
            if (error.status === 500) {
                console.error('Error interno del servidor', error);
            }
            return throwError(() => error);
        })
    );
};
```

Registrar los interceptors

Los interceptors se registran en el archivo `app.config.ts`. El orden importa: se ejecutan en el mismo orden en que se registran, igual que el pipeline de middleware en .NET.

app.config.ts

```
export const appConfig: ApplicationConfig = {
    providers: [
        provideRouter(routes),
        provideHttpClient(
            withInterceptors([
                authInterceptor,   // 1ro: agrega el JWT
                errorInterceptor   // 2do: captura errores
            ])
        )
    ]
};
```

Registro en .NET (Program.cs)	Registro en Angular (app.config.ts)
app.UseMiddleware<ExceptionMiddleware>(); app.UseAuthentication(); app.UseAuthorization();	provideHttpClient(withInterceptors([authInterceptor, errorInterceptor]))

04 · Guards: proteger rutas en Angular

¿Qué es un guard?

Un **guard** es una función que Angular ejecuta **antes de activar una ruta**. Si el guard retorna `true`, la navegación continúa. Si retorna `false` (o redirige a otra ruta), la navegación se cancela. Es el equivalente directo del atributo `[Authorize]` en .NET.

Concepto	.NET (Backend)	Angular (Frontend)
Proteger un recurso	[Authorize] en el controlador	canActivate en la ruta
Sin autenticación	Retorna 401 Unauthorized	Redirige a /login
Sin permisos	Retorna 403 Forbidden	Redirige a /sin-permiso
Acceso público	[AllowAnonymous]	No aplicar guard en la ruta

Guard de autenticación

guards/auth.guard.ts

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
    const authService = inject(AuthService);
    const router = inject(Router);

    if (authService.isAuthenticated()) {
        return true;  // tiene token -> puede pasar
    }

    // No tiene token -> redirigir al login
    router.navigate(['/login']);
    return false;
};
```

Aplicar el guard en las rutas

El guard se aplica directamente en la configuración de rutas. Las rutas sin guard son públicas (equivalente a `[AllowAnonymous]`), las rutas con guard requieren autenticación (equivalente a `[Authorize]`).

app.routes.ts

```
export const routes: Routes = [
    // Rutas públicas (sin guard = AllowAnonymous)
    { path: 'login', component: LoginComponent },
    { path: 'home',  component: HomeComponent },

    // Rutas protegidas (con guard = [Authorize])
    {
        path: 'productos',
        component: ProductosComponent,
        canActivate: [authGuard]   // requiere token
    },

    // Redirección por defecto
    { path: '', redirectTo: 'home', pathMatch: 'full' },
    { path: '**', redirectTo: 'home' }   // ruta no encontrada
];
```

> **El flujo completo:** el usuario intenta acceder a /productos → el guard ejecuta `authService.isAuthenticated()` → si no hay token, redirige a /login → el usuario se loguea → el AuthService guarda el JWT → el guard ahora retorna true → el usuario ve /productos → cada request que sale lleva el JWT gracias al interceptor.

05 · Pipes: transformar datos en el template

¿Qué es un pipe?

Un **pipe** es una función que transforma un valor en el template HTML antes de mostrarlo. Se usa con el operador `|` (pipe). Angular trae varios pipes incorporados y también permite crear pipes personalizados.

Pipes incorporados de Angular

Pipe	Qué hace	Ejemplo de uso	Resultado
date	Formatea fechas	{{ fecha | date:'dd/MM/yyyy' }}	25/06/2026
currency	Formatea moneda	{{ precio | currency:'ARS' }}	$1,500.00
uppercase	Todo a mayúsculas	{{ nombre | uppercase }}	NOTEBOOK
lowercase	Todo a minúsculas	{{ nombre | lowercase }}	notebook
titlecase	Primera letra mayúscula	{{ nombre | titlecase }}	Notebook
number	Formatea números	{{ total | number:'1.2-2' }}	1,500.00
percent	Formatea porcentaje	{{ 0.21 | percent }}	21%
json	Muestra objeto como JSON	{{ objeto | json }}	{ "id": 1 ... }

Usando pipes en un componente

template de un componente

```
@Component({
    standalone: true,
    imports: [CommonModule, CurrencyPipe, DatePipe],
    template: `
        <div *ngFor="let p of productos">
            <h3>{{ p.nombre | titlecase }}</h3>
            <p>Precio: {{ p.precio | currency:'ARS':'symbol':'1.2-2' }}</p>
            <p>Creado: {{ p.fechaCreacion | date:'dd/MM/yyyy HH:mm' }}</p>
        </div>`
})
export class ListaProductosComponent {
    productos = [
        { nombre: 'notebook', precio: 1500, fechaCreacion: new Date() },
        { nombre: 'teclado',  precio: 200,  fechaCreacion: new Date() }
    ];
}
```

Pipe personalizado

Podés crear tus propios pipes para transformaciones específicas de tu aplicación. Un pipe personalizado es una clase con el decorador `@Pipe` que implementa el método `transform()`.

pipes/estado-stock.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'estadoStock', standalone: true })
export class EstadoStockPipe implements PipeTransform {
    transform(cantidad: number): string {
        if (cantidad === 0) return 'Sin stock';
        if (cantidad < 10)  return 'Últimas unidades';
        return 'Disponible';
    }
}

// Uso en el template:
// <p>{{ producto.stock | estadoStock }}</p>
// Si stock = 3 -> muestra "Últimas unidades"
```

06 · Flujo completo: todo junto

Veamos cómo todas las piezas trabajan juntas en un escenario real: un usuario que se loguea y después consulta la lista de productos.

Paso	Capa	Qué sucede
1	**Usuario**	Ingresa email y contraseña en el formulario de login
2	**LoginComponent**	Llama a authService.login(credentials)
3	**AuthService**	Hace POST /api/auth/login con las credenciales
4	**Interceptor (auth)**	No agrega JWT porque todavía no hay token
5	**API .NET**	Valida credenciales, genera JWT y lo retorna
6	**AuthService**	Recibe el JWT y lo guarda en localStorage
7	**LoginComponent**	Redirige a /productos con router.navigate()
8	**Guard (auth)**	Verifica isAuthenticated() → true → permite acceder
9	**ProductosComponent**	Llama a productoService.getAll()
10	**Interceptor (auth)**	Detecta el JWT en localStorage y lo agrega al header
11	**API .NET**	Valida el JWT → [Authorize] permite el acceso
12	**API .NET**	Retorna la lista de productos con status 200
13	**Interceptor (error)**	No hay error → deja pasar la response
14	**ProductosComponent**	Recibe los productos y los muestra con pipes de formato

> **Conexión con el backend:** los pasos 5 y 11 son exactamente UseAuthentication + UseAuthorization + Controlador del pipeline de .NET. El interceptor de Angular es el espejo del middleware de .NET.

07 · Resumen comparativo y conceptos clave

Backend vs Frontend

Esta tabla resume cómo cada concepto que ya conocen del backend tiene su equivalente en Angular:

Responsabilidad	.NET (Backend)	Angular (Frontend)
Generar/Guardar JWT	El endpoint de login genera el token	El AuthService lo guarda en localStorage
Enviar JWT en cada request	El cliente lo envía en el header	El interceptor lo agrega automáticamente
Validar JWT	UseAuthentication() en el pipeline	La API lo valida (no Angular)
Proteger recursos	[Authorize] en el controlador	canActivate guard en la ruta
Acceso público	[AllowAnonymous]	Ruta sin guard
Manejar errores globales	ExceptionMiddleware	Error interceptor con catchError
Pipeline de procesamiento	Middleware en Program.cs	Interceptors en app.config.ts
Formatear datos	Serialización JSON automática	Pipes en el template
Validar datos de entrada	Data Annotations + ModelState	Validators en Reactive Forms

Resumen de conceptos clave

Concepto	Definición rápida
AuthService	Servicio que centraliza login, logout, guardar y obtener el JWT
localStorage	Almacenamiento del navegador donde se guarda el JWT entre sesiones
Interceptor	Función que procesa cada request HTTP antes de que salga de Angular
authInterceptor	Interceptor que agrega el header Authorization: Bearer automáticamente
errorInterceptor	Interceptor que captura errores HTTP (401, 403, 500) de forma global
req.clone()	Método para crear una copia modificada de un HttpRequest inmutable
Guard	Función que decide si una ruta puede activarse o no
canActivate	Tipo de guard que se ejecuta antes de entrar a una ruta

Concepto	Definición rápida
CanActivateFn	Tipo de función guard en Angular moderno (functional guard)
Pipe	Función que transforma un valor en el template antes de mostrarlo
@Pipe()	Decorador para crear un pipe personalizado
transform()	Método que implementa la lógica de transformación del pipe
provideHttpClient()	Función que registra HttpClient con los interceptors en app.config.ts
withInterceptors()	Función que registra la lista de interceptors funcionales